

里程碑

从零写出一个完整的 MPU-6050 IIO 驱动

在 VM 上用 i2c-stub 模拟,一步步手写并验证

配套仓库 github.com/fyy0619-cell/xiangmu · 2026 年 9 月

一、我做成了什么

从一个 hello 内核模块出发,从零手写出了一个结构完整、符合 Linux IIO 标准的 MPU-6050 传感器驱动,并在虚拟机上用 i2c-stub 模拟验证通过:用户 `cat` 一个文件就能拿到标准物理量。

最终功能

方面	实现
总线 / 匹配	i2c 驱动; <code>id_table</code> + 设备树 <code>of_match (compatible)</code>
探测 probe	regmap 建通道 → 读 WHO_AM_I 认芯片 → 唤醒(清 sleep)→ 注册 IIO
通道	加速度 X/Y/Z + 陀螺 X/Y/Z + 温度
数据接口	每通道 <code>raw</code> + <code>scale</code> ;温度另有 <code>offset</code>
读取回调	<code>read_raw</code> 按通道类型选寄存器、按 <code>mask</code> 返回 raw/scale/offset
资源管理	全程 <code>devm_</code> ,自动释放,无需 remove

代码见仓库 `examples/learning-driver/mpu6050_iio.c` 。

走过的步骤

hello 模块 → i2c 驱动骨架 → i2c-stub 触发 probe → regmap + WHO_AM_I → 唤醒 + 注册 IIO → 加 accel 通道 + `read_raw` → 加 gyro/temp 通道 → 加 scale/offset。 每一步都"改一点 → 编译加载 → 验证"。

二、read_raw 到底干嘛、为什么非要它(重点)

这是最容易糊涂、也最关键的一个点。一句话:**read_raw** 是"真正去芯片里把数值取出来"的那个函数。框架负责接待客人,但它不会读你这颗芯片——只有你写的 **read_raw** 会。

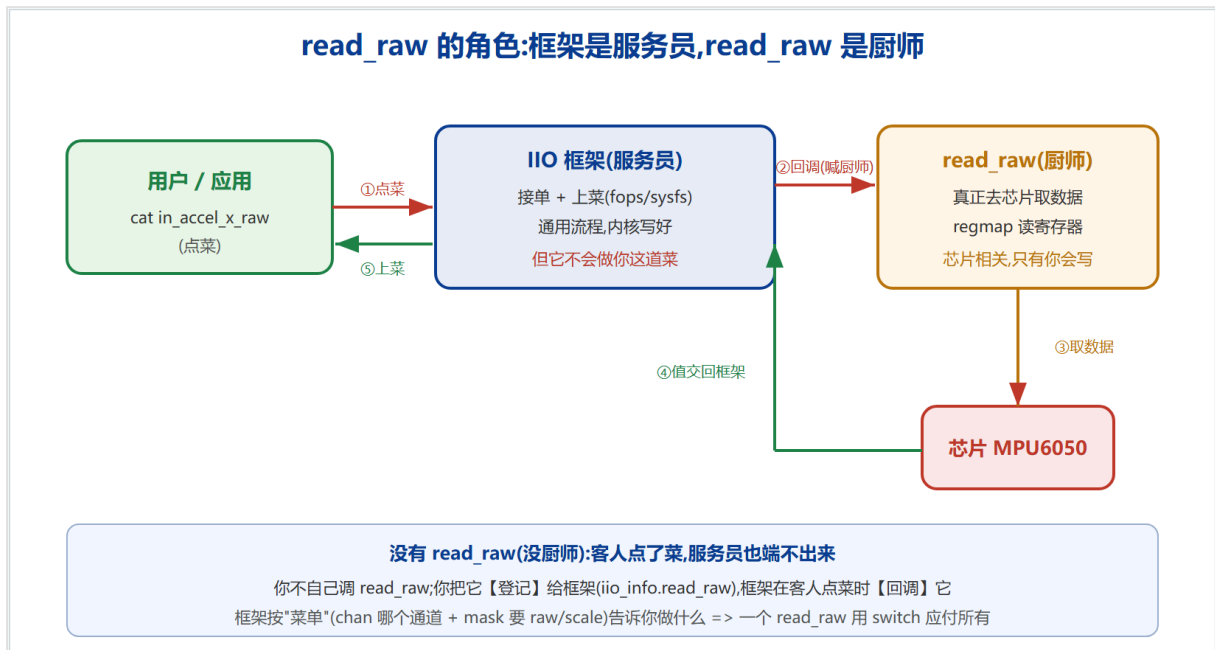


图 1 read_raw 的角色:框架是服务员,read_raw 是厨师

- **IIO 框架 = 服务员**:负责"客人点菜(cat 属性)"和"上菜(格式化返回)". 这套通用流程(fops/sysfs)内核都写好了。
- **read_raw = 厨师**:真正做菜——去 MPU6050 的寄存器读出数值。框架不会做你这道菜(它不懂你芯片的寄存器),只有你会。
- **没有 read_raw 会怎样**:属性文件还在,但用户 cat 时框架没厨师可喊,端不出任何数据。read_raw 是数值的唯一来源。
- **为什么是回调**:你不自己调它,而是把它登记给框架(iio_info.read_raw);用户一点菜,框架就回调它(控制反转)。
- **参数就是点菜单**:框架传 chan (哪个通道)和 mask (要 raw 还是 scale),你照单取值写回 *val。所以一个 read_raw 用 switch 就能应付所有通道、所有类型。

```
static int mpu6050_read_raw(indio_dev, chan, *val, *val2, mask) {
    data = iio_priv(indio_dev);          // 取回私有数据(拿 regmap)
    switch (mask) {                      // 内核要哪类值?
        case RAW: 按 chan->type 选寄存器 → regmap 读 → *val = ... ; return IIO_VAL_INT;
        case SCALE: 按 chan->type 返回换算系数;
        case OFFSET: 返回温度偏置;
    }
}
```

记住:框架管"接待和上菜"(通用,内核写好);read_raw 管"做这道菜"(芯片相关,只有你会)。没有它,客人点了菜却没人做。

三、数据流与物理换算

```
cat in_accel_x_raw
→ IIO 核心(服务员)
→ 回调你的 read_raw(厨师)
→ regmap 读芯片 0x3B/0x3C
→ 返回原始值 → 内核格式化 → 用户看到数字
```

驱动只给**原始值**和**换算系数**,真正的单位换算由用户态按 IIO 约定完成:

- 加速度(m/s²) = `in_accel_x_raw × in_accel_scale`
- 角速度(rad/s) = `in_anglvel_x_raw × in_anglvel_scale`
- 温度(°C) = `(in_temp_raw + in_temp_offset) / 340`

实测(假芯片预置 1g 时): `in_accel_z_raw = 16384`, × `0.000598550` ≈ **9.806 m/s²**,正好是一个重力加速度。

四、我真正掌握了什么

概念	我已亲手实现/理解
设备-驱动匹配	驱动待命 → 总线出现设备 → 内核调 probe
probe 的三件事	分配(iio_dev+私有) → 初始化(regmap/认芯片/唤醒) → 开接口(注册+通道+read_raw)
私有数据	<code>iio_priv</code> 两处取同一块:probe 存 regmap,read_raw 取
通道与 read_raw	多类型通道按 type 选寄存器,按 mask 返回 raw/scale/offset
框架分工	fops/sysfs 在内核核心,我只写芯片相关的 read_raw
工程习惯	骨架优先、增量填肉、每步编译验证;大括号配对排错

学习版代码: `examples/learning-driver/` ;完整从零教程见 `docs/learning/ 01-08`。配套仓库:github.com/fyy0619-cell/xiangmu